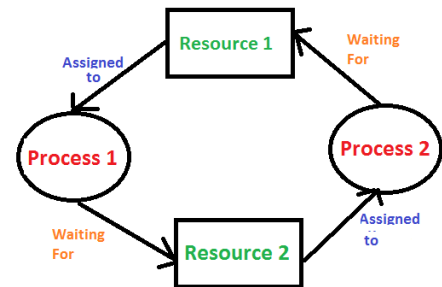


DEADLOCK

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.



Deadlock characterization

For a deadlock to occur, four conditions must be simultaneously true:

Mutual Exclusion: At least one resource must be held in a non-sharable mode; only one process can use the resource at any given time.

Hold and Wait: A process is holding at least one resource and waiting for resources that are currently being held by other processes.

No Preemption: A resource cannot be taken from a process unless the process releases the resource.

Circular Wait: A set of processes are waiting for each other in a circular chain, such that each process is waiting for a resource that the next process in the chain holds.

Deadlock Prevention

Deadlock prevention involves designing the system in such a way that at least one of the necessary conditions for deadlock cannot hold. Here are the methods for preventing each of the four conditions:

Mutual Exclusion: Ensure that some resources, if possible, are sharable. For example, read-only files can be shared among multiple processes. However, some resources are inherently non-sharable (e.g., a printer).

Hold and Wait: Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated

No Preemption: Preempt resources from the process when resources are required by other high-priority processes.

Circular Wait: Each resource will be assigned a numerical number. A process can request the resources to increase/decrease. order of numbering. For Example, if the P1 process is allocated R5 resources, now next time if P1 asks for R4, R3 lesser than R5 such a request will not be granted, only a request for resources more than R5 will be granted.

Deadlock Avoidance

Deadlock avoidance algorithms dynamically examine the resource-allocation state to ensure that a circular wait condition can never exist. One common method is: Banker's Algorithm.

The Banker's Algorithm works by simulating resource allocation for processes in advance to ensure that it doesn't lead to a deadlock and checking if the system is remain in a safe state.

Detection and Recovery:

Another approach to dealing with deadlocks is to detect and recover from them when they occur. The system can use resource allocation graphs and algorithms to detect cycles (circular waits) in the resource allocation graph. Once a deadlock is detected, the system must take action to break the deadlock. This can be done by:

Terminating processes: Abort one or more processes to break the circular wait.

Resource Preemption: Temporarily take resources away from some processes and give them to others until the deadlock cycle is broken.

Banker's algorithm

The Banker's Algorithm works by simulating resource allocation for processes in advance to ensure that it doesn't lead to a deadlock and checking if the system is remain in a safe state.

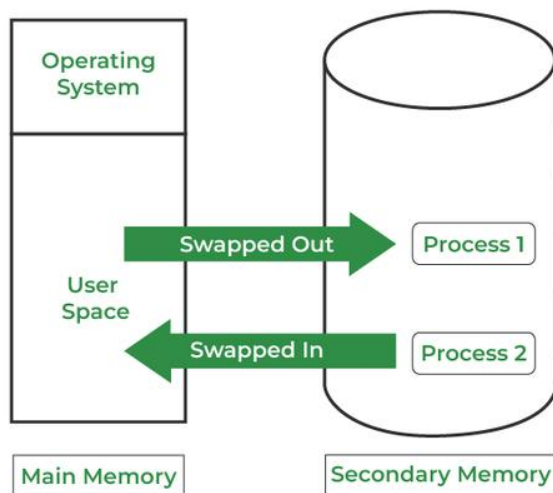
XXX

Memory management

Allocate and de-allocate memory before and after process execution.

Swapping:

Moving entire processes in and out of main memory to disk storage. the memory manager can swap out the lower priority process and then load and execute the higher priority process. After finishing higher priority work, the lower priority process swapped back in memory and continued to the execution process.



Contiguous Memory Allocation:

In contiguous memory allocation, each process is allocated a single continuous block of memory.

Fixed Partition Allocation:

Memory is divided into fixed-size partitions at the time of system startup.

Each partition can hold exactly one process.

The number and size of partitions do not change during system operation.

Advantages:

Simplicity: Easy to implement and manage.

Predictability: Memory allocation and deallocation are straightforward and predictable.

Disadvantages:

Internal Fragmentation: If a process is smaller than the partition size, the remaining space in the partition is wasted.

Limited Flexibility: Fixed number of partitions means a limited number of processes can be in memory simultaneously.

Inefficiency: Processes may be queued waiting for an available partition, even if there is enough free memory overall.

XXX

Variable Partition Allocation

Memory is allocated to processes dynamically, based on their size requirements.

Partitions are created on-the-fly as processes are loaded into memory and are sized to fit the process.

The size and number of partitions can change during system operation.

Advantages:

Reduced Fragmentation: Better utilization of memory with reduced internal fragmentation.

Flexibility: Can accommodate varying sizes of processes without predefined limits.

Efficient Memory Use: More processes can be accommodated by fitting partitions closely to process sizes.

Disadvantages:

External Fragmentation: Free memory can become fragmented over time, making it difficult to allocate contiguous memory blocks to new processes.

Complexity: More complex to manage due to the need for dynamic allocation and deallocation.

Overhead: Additional overhead for maintaining data structures that track free and used memory blocks.

XXX

Non-Contiguous Allocation:

Memory is allocated in various non-contiguous blocks, helping to reduce fragmentation.

Paging:

Divides memory into fixed-size pages and divides programs into pages that can be loaded into any memory frame.

Page Table:

A data structure used to keep track of the mapping between virtual addresses and physical addresses.

Each process has its own page table that translates virtual addresses to physical addresses.

Physical address space:

- Refers to the actual addresses in the physical memory.
- The page number is used as an index into the page table to find the corresponding frame number.
- The frame number is then combined with the offset to form the physical address in memory.

logical address space:

- A logical address, also known as a virtual address, is an address generated by the CPU during program execution.
- **Page Number:** Identifies the page in the logical address space.
- **Offset:** Identifies the specific location within that page.

Advantages

Eliminates External Fragmentation: No need for contiguous memory allocation.

Efficient Memory Use: Pages can be loaded into any available frame.

Easy Swapping: Pages can be easily swapped between physical memory and disk.

Disadvantages

Overhead: Maintaining page tables and TLBs incurs overhead.

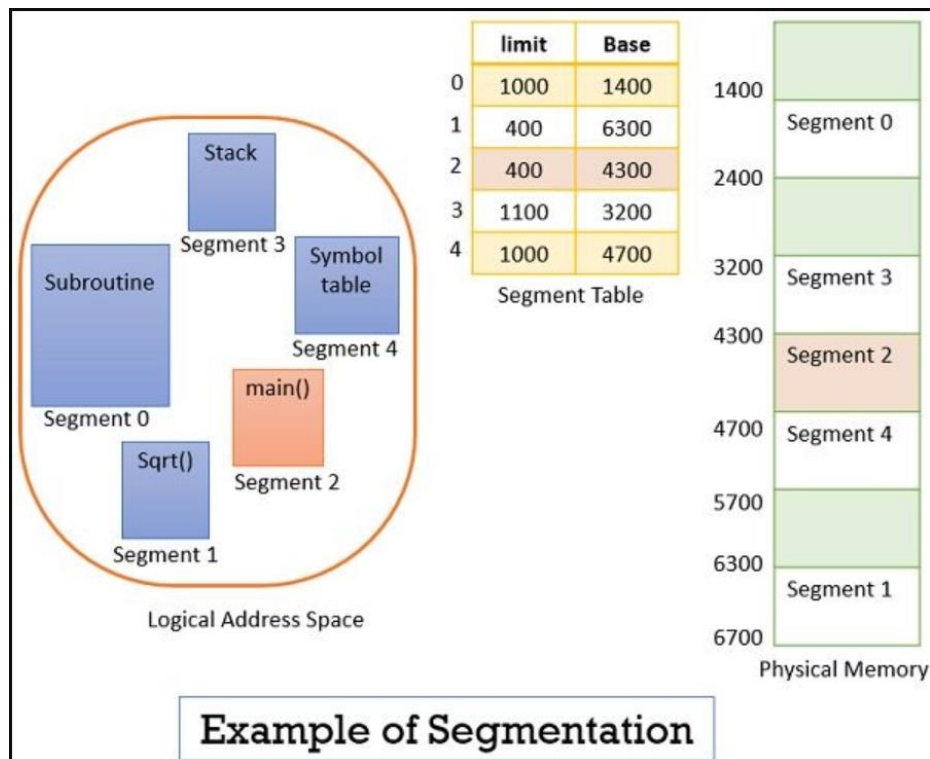
Page Faults: Occur when a referenced page is not in memory, causing delays as the page is loaded from disk.

Segmentation

Segmentation in operating systems (OS) refers to a memory management technique where a process's memory is divided into different segments, each of which can be a different size and can correspond to a different part of the process, such as code, data, stack, etc.

Segment Table:

- The operating system maintains a segment table for each process.
- Each entry in the segment table has a base address (indicating where the segment begins in physical memory) and a limit (indicating the segment's length).
- The segment number is an index into the segment table to get the base address and the limit.
- The offset is added to the base address to get the physical address in memory.



Virtual memory:

Virtual memory is a memory management technique used by operating systems to give an application the impression that it has access to a large, contiguous block of memory, even if the physical memory (RAM) available is smaller.

Advantages of Virtual Memory

Isolation: Each process operates in its own virtual address space, providing protection and

isolation from other processes.

Efficient Memory Use: Allows the system to use physical memory more efficiently by loading only the needed parts of programs and data into RAM.

Larger Address Space: Enables programs to use more memory than is physically available by leveraging disk space.

Security: Enhances security by isolating processes and providing a controlled environment for memory access.

XXX

Page Faults:

Occurs when a program tries to access a page that is not currently in physical memory. The operating system then retrieves the page from the disk and loads it into a free frame in physical memory, updating the page table accordingly.

Demand Paging

The process of loading the page into memory on demand (whenever a page fault occurs) is known as demand paging.

Thrashing:

Thrashing is a phenomenon in computer systems, particularly in virtual memory environments, where the system spends a significant amount of time swapping pages in and out of memory rather than executing actual instructions.